Final Project Report

Project Title: Sniff–Then–Spoof ICMP Reply Attack in a Virtualized LAN

Students: Gilbert Ncube & Tinaishe Tapera

Course: Computer and Network Security

---

1. Abstract

This project demonstrates a real-world network-layer attack in which an attacker intercepts and forges ICMP traffic in a virtual LAN environment. Using Python and Scapy, we implemented a system that sniffs ICMP Echo Requests and immediately injects spoofed Echo Replies. The experiment highlights security weaknesses in ARP and ICMP, showing how unauthenticated protocols can be abused to mislead network hosts. Wireshark was used to capture and analyze legitimate versus spoofed packets. This work directly fulfills the project objectives outlined in the original proposal.

2. Network Topology & Environment Setup

2.1 Virtual Machine Configuration

Machine Role	Operating System	IP Address
Attacker VM	Kali Linux	192.168.56.102
Victim 1 (Sender)	Kali Linux	192.168.56.103
Victim 2 (Target)	Kali Linux	192.168.56.104

- All machines were connected to the **same host-only virtual network,** enabling ARP and ICMP traffic interception.
- No external internet access was required
- Tools used:

- Python 3
- Scapy
- Wireshark

3. Attack Implementation Overview

The attack was implemented in **three stages** using three Python scripts:

1. **ICMP Sniffer** – Captures live ICMP traffic
2. **Manual ICMP Spoofer** – Sends forged ICMP replies
3. **Final Automated Sniff-Then-Spoof Engine** – Performs ARP poisoning, live sniffing, and payload-preserving ICMP spoofing

4. Script Breakdown

4.1 ICMP Packet Sniffer

This script captures and displays all ICMP packets in real time.

sniff(iface="eth0", filter="icmp", prn=show_packet)

Confirms ICMP traffic between victims
Displays packet structure
Verifies ID, sequence numbers, and payload

4.2 Manual ICMP Spoofing Tool

This script sends a forged ICMP Echo Reply to the victim.

ip = IP(src=pretend_to_be, dst=victim_ip)

icmp = ICMP(type=0)

packet = ip/icmp

send(packet)

Demonstrates basic spoofing
Victim accepts fake replies from nonexistent hosts
Confirms ICMP has **no authentication**

4.3 Final Automated Sniff–Then–Spoof Attack Script

This script performs:

- ARP poisoning
- Live ICMP interception
- Payload-preserving ICMP spoofed replies

Key features:

- Matches ICMP ID and sequence numbers
- Preserves payload size (fixes “truncated” ping issue)
- Sends fake replies faster than real hosts

This script represents the full **Man-In-The-Middle ICMP spoofing attack**

5. How to Run the Demo (Step-By-Step)

Step 1: Start Wireshark on Attacker VM

On **Kali (Attacker)**:

sudo wireshark

Select interface:

eth0

Apply this filter:

icmp || arp

Step 2: Start the Final Attack Script (Attacker VM)

sudo python3 final_attack.py -i eth0 -v

Expected output:

[*] Starting sniffing on eth0...

[*] Performing both ARP poisoning and ICMP spoofing

Step 3: Start Ping from Victim 1

On **Victim 1 (192.168.56.103)**:

ping 192.168.56.104

Step 4: Observe Spoofed Replies

Victim 1 will immediately begin receiving replies:

64 bytes from 192.168.56.104: icmp_seq=1 ttl=64 time=0.6 ms

64 bytes from 192.168.56.104: icmp_seq=2 ttl=64 time=0.7 ms

Replies continue even if Victim 2 is shut down
This proves the replies are fake

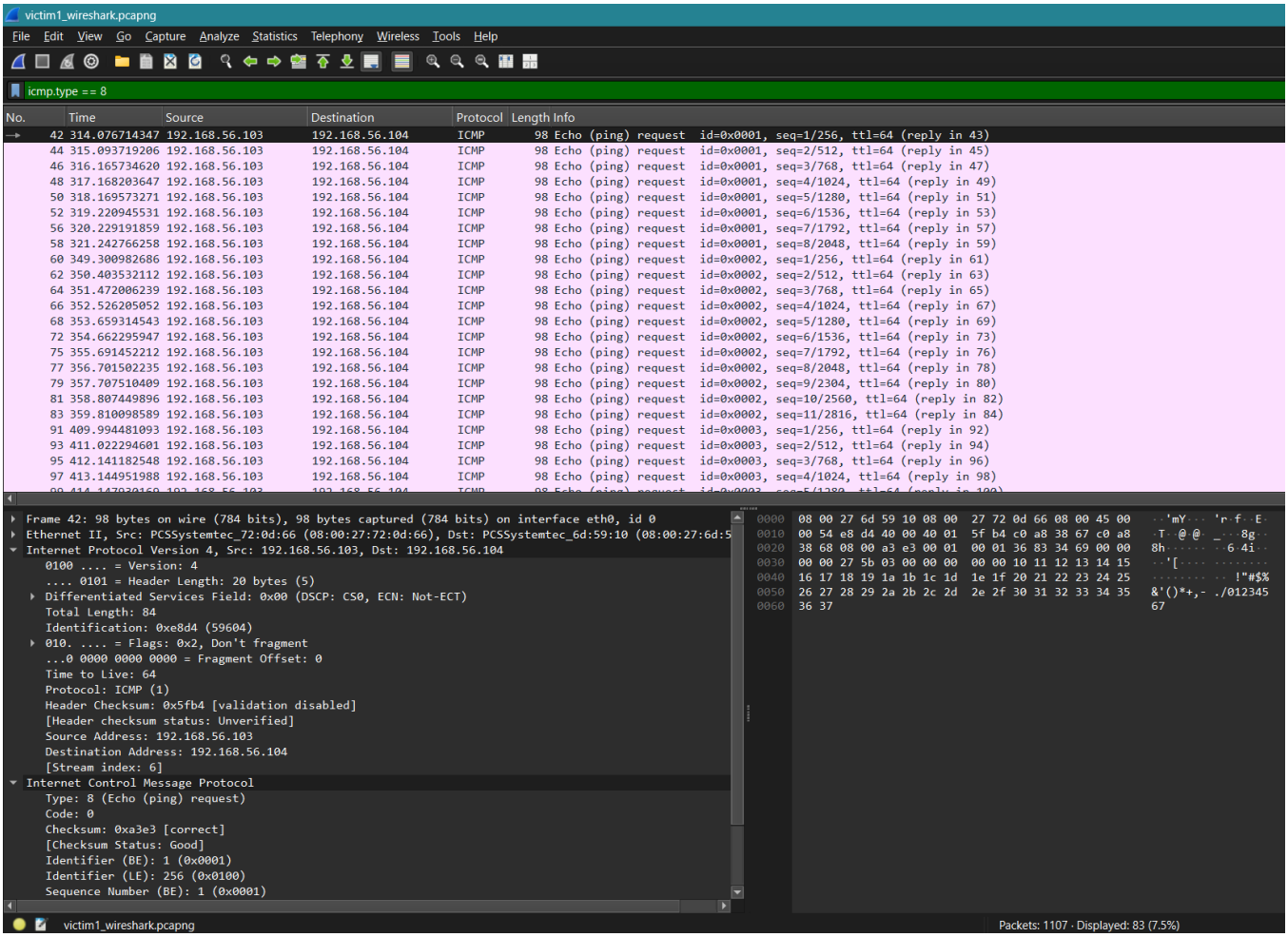
6. Wireshark Packet Capture & Analysis

Wireshark Filter Used

icmp || arp

Screenshot 1: ICMP Echo Request from Victim 1

Here apply filter **“icmp.type == 8”**



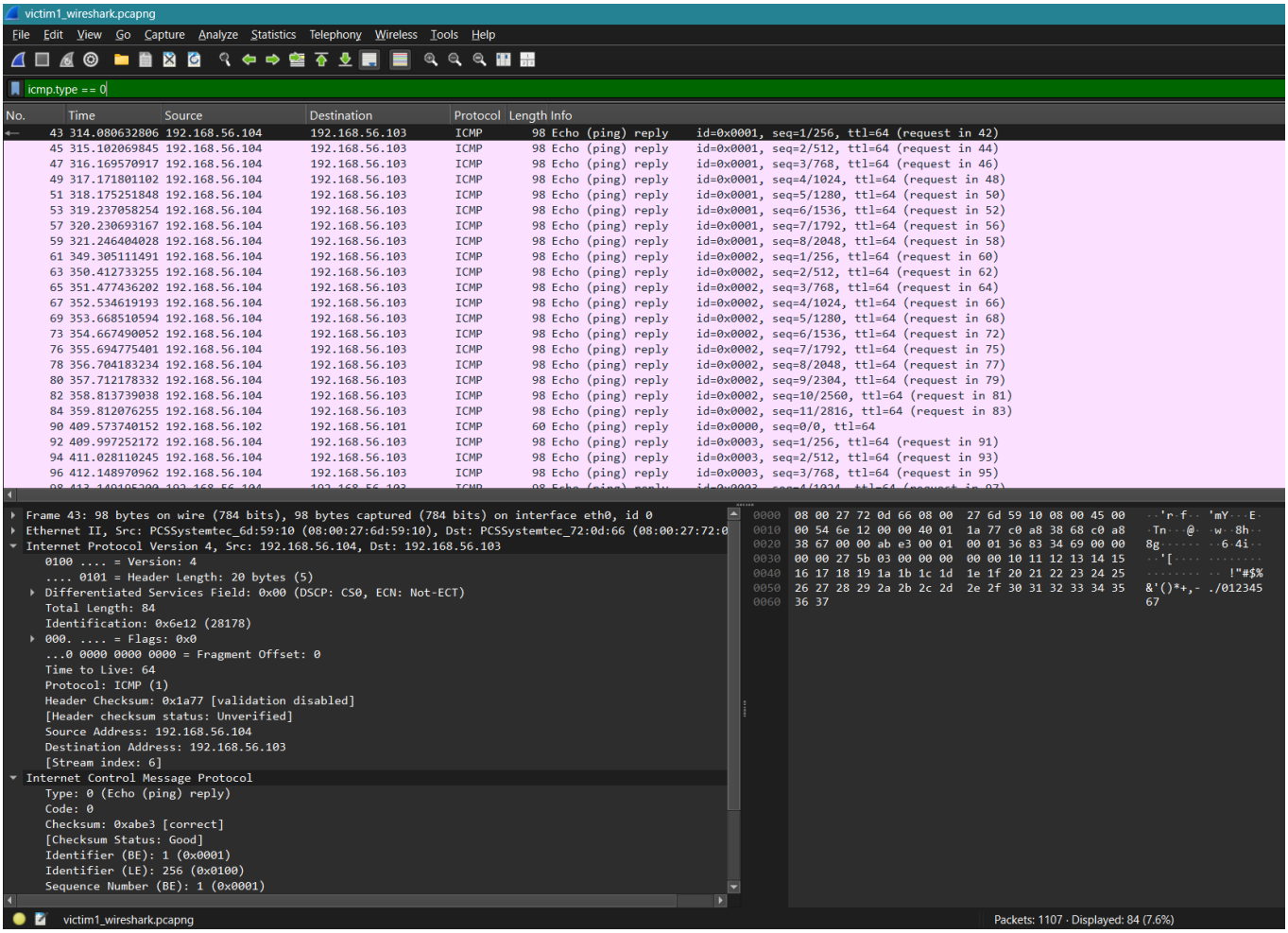
Observed Fields:

- Source IP: 192.168.56.103
- Destination IP: 192.168.56.104
- ICMP Type: 8 (Echo Request)
- Sequence Number: increments normally

This confirms the legitimate ping request.

Screenshot 2: Spoofed ICMP Echo Reply from Attacker

Here apply filter "**icmp.type == 0**"



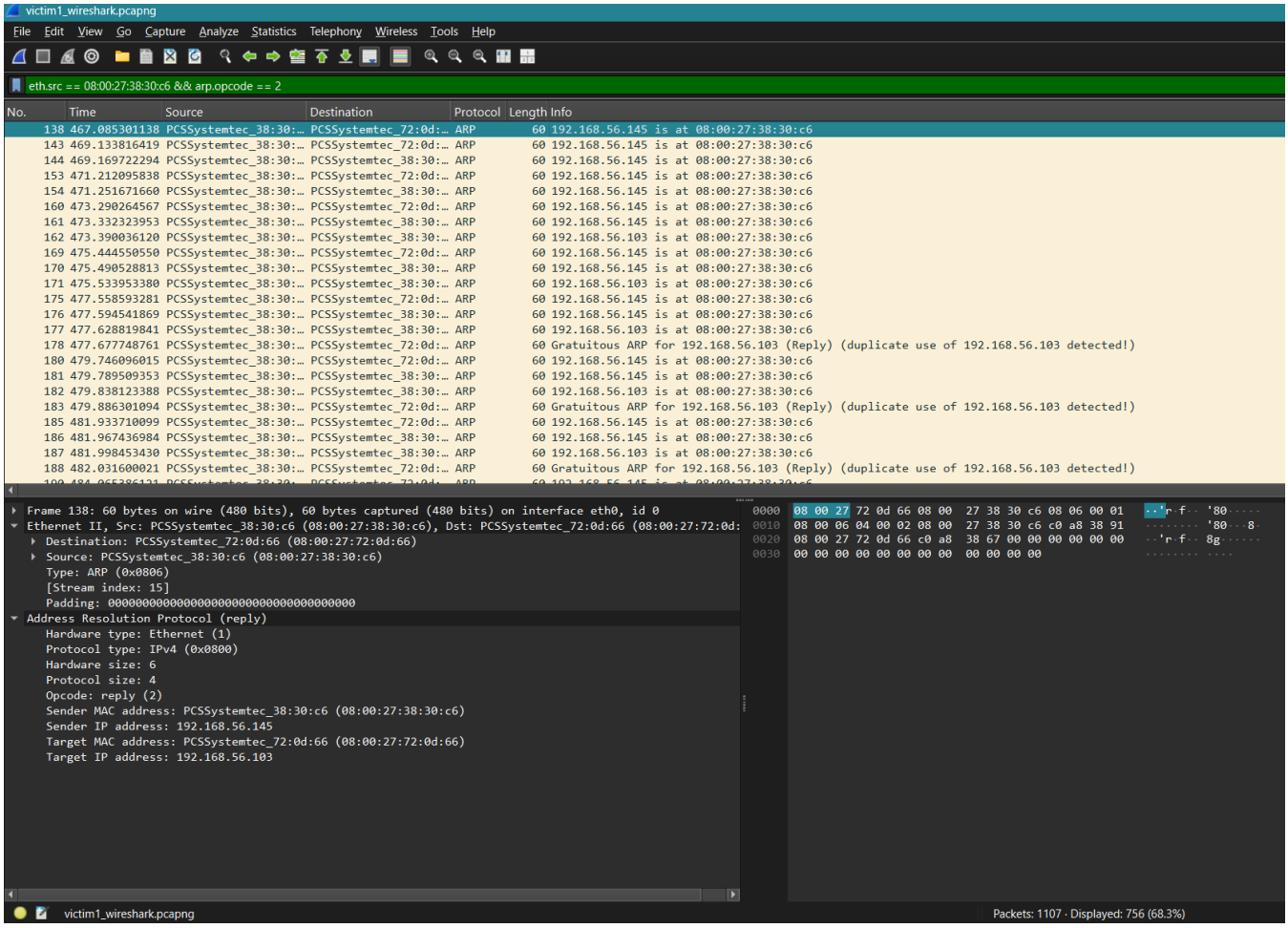
Observed Fields:

- Source IP: 192.168.56.104 (spoofed)
- Destination IP: 192.168.56.103
- ICMP Type: 0 (Echo Reply)
- Same Sequence Number as request
- Same payload length

This confirms the attacker successfully forged a valid reply.

Screenshot 3: ARP Poisoning Packet

Here apply filter "**eth.src == 08:00:27:38:30:c6 && arp.opcode == 2"** for ARP replies sent by attacker MAC



Observed Fields:

- ARP Operation: Reply (op = 2)
- Fake MAC mapped to victim IP

This confirms the attacker became Man-In-The-Middle. Here the attacker:

- Listens for ARP "who-has" requests.
- Sends forged ARP replies claiming that the requested IP is at the attacker's MAC address (08:00:27:38:30:c6).
    - **Spoofed packets**: Source IP doesn't match any real machine

7. Security Analysis

This experiment demonstrates several critical weaknesses:

Vulnerability	**Demonstrated**
ICMP has no authentication	✓
ARP has no trust verification	✓
MITM possible on LAN	✓
Source IP spoofing	✓
Victims blindly trust replies	✓

Without firewall rules, IDS, or encrypted tunnels, a local attacker can fully manipulate ICMP communication.

8. Limitations

- Works only on local Layer-2 networks
- Does not bypass encrypted protocols
- Easily detectable by IDS/IPS
- ICMP filtering would block this attack

9. Conclusion

This project successfully demonstrates a complete sniff-then-spoof ICMP attack in a controlled virtual environment. By exploiting weaknesses in ARP and ICMP, an attacker was able to intercept live traffic and inject forged Echo Replies that were trusted by the victim machine. Through Wireshark analysis, we confirmed that spoofed packets matched legitimate traffic fields, proving that unauthenticated network-layer protocols remain vulnerable. This project highlights why modern networks require intrusion detection, packet filtering, and encryption for protection.